

Übungsblatt 4: Race Conditions, Mutex und Priority Inversion

Musterlösung · Echtzeitsysteme (EZS)

Prof. Dr.-Ing. Daniel Klünder

Musterlösung

Zielsetzung

In dieser Laborübung machen Sie die Gefahren von ungeschützten Zugriffen auf geteilte Ressourcen (*Shared Resources*) sichtbar. Sie erzeugen zunächst eine *Race Condition* am seriellen Port (UART) und beheben diese anschließend mithilfe eines CMSIS-Mutexes. Danach provozieren Sie absichtlich eine *Priority Inversion* — das klassische Fehlerbild der Mars-Pathfinder-Mission — und beweisen, dass der Mutex das System durch *Priority Inheritance* (Prioritätsvererbung, PIP) retten kann.

Aufgabe 1: Das Chaos am UART (Race Condition)

Bauen Sie ein System, in dem zwei Tasks asynchron auf dieselbe Hardware-Ressource (den UART / `printf()`) zugreifen.

1. Legen Sie im CubeMX-Configurator zwei Tasks an:
 - Task1_Normal (Priorität: `osPriorityNormal`)
 - Task2_High (Priorität: `osPriorityHigh`)
2. Lassen Sie beide Tasks in ihren jeweiligen `for(;;)`-Schleifen einen langen Text (z. B. "Task 1 sendet in Ruhe Daten...\n") via `printf()` ausgeben. Fügen Sie anschließend ein `osDelay(100)` in beide Tasks ein.
3. Verwenden Sie **keinerlei Absicherung** (weder Mutex noch Semaphor).
4. Flashen Sie den Code und beobachten Sie die serielle Konsole Ihres PCs. Was fällt Ihnen an der Ausgabe auf? Erklären Sie die Ursache.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 1

Beispielcode (ungeschützt):

```

/* USER CODE BEGIN Header_StartTask1_Normal */
void StartTask1_Normal(void *argument)
{
    for (;;) {
        printf("Task 1 sendet in Ruhe Daten...\n");
        osDelay(100);
    }
}
/* USER CODE END Header_StartTask1_Normal */

/* USER CODE BEGIN Header_StartTask2_High */
void StartTask2_High(void *argument)
{
    for (;;) {
        printf("Task 2 sendet in Ruhe Daten...\n");
        osDelay(100);
    }
}
/* USER CODE END Header_StartTask2_High */

```

Beobachtung auf der Konsole:

Die Konsole zeigt zerrissenen String-Salat („Buchstabensalat“). Textteile von Task 1 und Task 2 sind wild ineinander verschachtelt.

Erklärung:

Das Verhalten ist eine klassische *Race Condition*. Der Aufruf von `printf()` schreibt die Zeichen nacheinander (bytwiese) in das UART-Register. Da `Task2_High` eine höhere Priorität hat, präemptiert (unterbricht) sie `Task1_Normal` durch den SysTick mitten im Schreibvorgang. `Task2_High` schreibt ihren Text in das UART-Register, legt sich mit `osDelay` schlafen, und `Task1_Normal` schreibt danach den Rest ihres Textes zu Ende. Die geteilte Ressource UART ist nicht atomar geschützt.

Aufgabe 2: Die Absicherung (Mutex)

Wir reparieren das Chaos aus Aufgabe 1.

1. Legen Sie im Code (z. B. vor den Task-Funktionen) einen Mutex an: `uartMutex = osMutexNew(NULL);`.
2. Umschließen Sie den `printf()`-Aufruf in **beiden** Tasks mit `osMutexAcquire()` (Timeout: `osWaitForever`) und `osMutexRelease()`.
3. **Achtung:** Das `osDelay()` muss **außerhalb** des Mutex-Blocks (der kritischen Sektion) stehen! Andernfalls halten Sie den Mutex während der Wartezeit und blockieren die andere Task unnötig lange.
4. Flashen Sie den Code und überprüfen Sie die Konsolenausgabe.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 2

Code-Implementierung:

```
/* USER CODE BEGIN 0 */
osMutexId_t uartMutex;
/* USER CODE END 0 */

/* In main() bzw. USER CODE BEGIN 2: */
uartMutex = osMutexNew(NULL);

/* USER CODE BEGIN Header_StartTask1_Normal */
void StartTask1_Normal(void *argument)
{
    for (;;) {
        osMutexAcquire(uartMutex, osWaitForever);
        printf("Task 1 sendet in Ruhe Daten...\n");
        osMutexRelease(uartMutex);
        osDelay(100); /* bewusst AUSSERHALB der kritischen Sektion */
    }
}
/* USER CODE END Header_StartTask1_Normal */

/* USER CODE BEGIN Header_StartTask2_High */
void StartTask2_High(void *argument)
{
    for (;;) {
        osMutexAcquire(uartMutex, osWaitForever);
        printf("Task 2 sendet in Ruhe Daten...\n");
        osMutexRelease(uartMutex);
        osDelay(100); /* bewusst AUSSERHALB der kritischen Sektion */
    }
}
/* USER CODE END Header_StartTask2_High */
```

Ergebnis:

Die Strings im Terminal sind nun sauber und ungestört. Der Mutex schützt die kritische Sektion (das `printf`). Versucht Task 2 zu schreiben, während Task 1 den Mutex hält, wird Task 2 vom Scheduler in den Zustand *Blocked* versetzt, bis Task 1 das Schloss freigibt.

Hinweis zur Platzierung von `osDelay`: Ließe man `osDelay` *innerhalb* von `Acquire/Release`, bliebe der Mutex während der gesamten Wartezeit gesperrt. Die andere Task könnte in dieser Zeit nicht auf den UART zugreifen — das verlängert die kritische Sektion unnötig und begünstigt später Prioritätsinversion.

Aufgabe 3: Die Sabotage (Priority Inversion)

Wir bauen nun den fatalen Fehler des Mars Pathfinder nach. Pathfinder litt unter Priority Inversion, weil Prioritätsvererbung (PIP) damals deaktiviert war. Im FreeRTOS-/CMSIS-Labor erreichen wir denselben Effekt didaktisch sauber, indem wir den Mutex durch einen **binären Semaphor** ersetzen — Semaphoren besitzen weder Ownership noch PIP.

1. Ersetzen Sie in Ihrem Code den Aufruf `osMutexNew` durch einen binären Semaphor:

```
uartSem = osSemaphoreNew(1, 1, NULL); /* MaxCount=1, InitialCount=1 */
```

Passen Sie Handle-Typ und API an (`osSemaphoreId_t`, `osSemaphoreAcquire` / `osSemaphoreRelease`). Das Schutzmuster bleibt gleich: `Acquire` → `printf` → `Release` → `osDelay` (Delay weiterhin außerhalb!).

2. Fügen Sie im Configurator eine dritte Task hinzu: `Task3_Medium` (Priorität: `osPriorityAboveNormal` — also zwischen Normal und High).
3. Schreiben Sie in `Task3_Medium` eine reine CPU-Fresser-Schleife (wie in der letzten Laborübung):

```
volatile uint32_t x = 0;
while (1) { x++; }
```

Nutzen Sie **kein** Delay und fassen Sie den Semaphor **nicht** an!

4. Flashen Sie den Code und beobachten Sie die Konsole.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 3

Code-Ausschnitte:

```
/* USER CODE BEGIN 0 */
osSemaphoreId_t uartSem;
/* USER CODE END 0 */

uartSem = osSemaphoreNew(1, 1, NULL);

/* In Task1_Normal und Task2_High: */
osSemaphoreAcquire(uartSem, osWaitForever);
printf("...");
osSemaphoreRelease(uartSem);
osDelay(100); /* weiterhin außerhalb! */

/* USER CODE BEGIN Header_StartTask3_Medium */
void StartTask3_Medium(void *argument)
{
    volatile uint32_t x = 0;
    for (;;) {
        x++; /* 100 % CPU-Last, kein Delay, kein Semaphor */
    }
}
```

```
}  
/* USER CODE END Header_StartTask3_Medium */
```

Beobachtung:

Das Terminal verstummt oder das System „hängt“. Es erscheinen keine (oder kaum noch) `printf()`-Ausgaben der `Task2_High` mehr, obwohl diese die höchste Priorität im System hat.

Erklärung der Priority Inversion (nicht Starvation!):

1. `Task1_Normal` sperrt den Semaphor und beginnt zu schreiben.
2. `Task2_High` wacht auf, möchte schreiben, scheitert am `Acquire` und wechselt in *Blocked*.
3. Nun wird `Task3_Medium` bereit. Da sie eine höhere Priorität als `Task1_Normal` hat, präemptiert sie `Task1_Normal`.
4. Da `Task3_Medium` in einer Endlosschleife hängt, kommt `Task1_Normal` nie wieder an die Reihe, um den Semaphor durch `Release` freizugeben.
5. **Konsequenz:** `Task2_High` wartet unbegrenzt auf eine Freigabe, die niemals erfolgt. Eine *mittelpriore* Task blockiert effektiv die *höchstpriore* Task — das ist **Priority Inversion**.

Abgrenzung zu Starvation: Starvation bedeutet, dass eine Task dauerhaft keine CPU-Zeit erhält, weil höherpriore Tasks die CPU dauerhaft belegen. Hier ist das Kernproblem anders: Die High-Task ist *blocked auf eine Ressource*, die von einer Low-Task gehalten wird, welche wiederum von einer Medium-Task präemptiert wird. Der Mechanismus heißt Priority Inversion (unbounded, weil die Medium-Task nie yieldet).

Da ein Semaphor in FreeRTOS/CMSIS **kein** Ownership und **kein** Priority Inheritance Protocol (PIP) unterstützt, greift das Betriebssystem nicht helfend ein — analog zum Pathfinder-Szenario vor dem NASA-Patch.

Hinweis zur Beobachtbarkeit: Ist die kritische Sektion sehr kurz, kann die Inversion flüchtig sein. Für die Demo dürfen Sie in Task 1 *innerhalb* von `Acquire/Release` kurz busy-warten (z. B. eine Zählschleife) — **nicht** `osDelay`, denn `osDelay` würde die Task blockieren und den Scheduler freiwillig abgeben.

Aufgabe 4: Die Heilung (Priority Inheritance)

Wir retten unser System durch eine einzige API-Änderung — analog zum NASA-Patch der Pathfinder-Mission.

1. Ersetzen Sie den fehlerhaften Semaphor im Code wieder durch den korrekten CMSIS-Mutex: `uartMutex = osMutexNew(NULL);`. Ändern Sie die Task-Logik der drei Tasks sonst nicht (`Acquire` → `printf` → `Release` → `osDelay` außerhalb; Medium-Task bleibt CPU-Fresser).
2. Flashen Sie den Code neu.
3. Erklären Sie, warum das System nun wieder funktioniert, obwohl `Task3_Medium` weiterhin 100 % CPU-Last fordert.

Ihre Lösung

(Notieren Sie Ihre Lösung hier.)

Musterlösung Aufgabe 4

Code-Änderung (Kern):

```
/* zurück zum Mutex mit Ownership + PIP */
osMutexId_t uartMutex;
uartMutex = osMutexNew(NULL);

/* Tasks 1 und 2 wieder mit Mutex-API: */
osMutexAcquire(uartMutex, osWaitForever);
printf("...");
osMutexRelease(uartMutex);
osDelay(100);
```

Beobachtung:

Das Terminal sendet wieder sauber Daten. Das System hat sich erholt.

Erklärung (Priority Inheritance Protocol, PIP):

Ein CMSIS-Mutex (im Gegensatz zum binären Semaphor) besitzt *Ownership* und hat das *Priority Inheritance Protocol* (PIP) aktiviert.

Ablauf:

1. Wenn **Task2_High** den von **Task1_Normal** gehaltenen Mutex anfordert, erkennt das RTOS die drohende Inversion.
2. Der Scheduler **boostet** die Priorität von **Task1_Normal** temporär auf die Priorität der wartenden High-Task.
3. Da **Task1_Normal** nun die Priorität High geerbt hat, kann **Task3_Medium** sie nicht mehr verdrängen.
4. **Task1_Normal** beendet ihre kritische Sektion (**printf**), ruft **Release** auf und fällt sofort wieder auf ihre ursprüngliche Priorität (**osPriorityNormal**) zurück.
5. **Task2_High** übernimmt den Mutex und schreibt ihre Daten.

Die *unbounded* Priority Inversion wurde durch das RTOS erfolgreich begrenzt: **Task2_High** wartet höchstens noch auf die Restlaufzeit der kritischen Sektion von **Task1_Normal**, nicht mehr auf die Endlosschleife von **Task3_Medium**. Diesen Schutzmechanismus (PIP) aktivierte die NASA nachträglich auch beim Pathfinder.